Lab 5

BCD Up/Down Counter on 7-segment display

By: Ben Robles & Robert Stevenson
Group U

ECE-3300L
Summer 2026

Date: 07/16/2026

Design:

clock_divider.v:

```
module clock_divider (
                    input wire clk,
                    output reg [31:0] clk_div
                  );

   initial clk_div = 0;

   always@(posedge clk)
   begin
      clk_div <= clk_div + 1;
   end

endmodule
```

Description:

This module creates a 32-bit counter that increments on posedge clk. This functions as a clock divider because each bit toggles at different freqencies in a way that higher bits toggle slower.

mux2x1.v:

```
module mux2x1(
              input wire sel,
              input wire a,
              input wire b,
              output wire y
            );

   wire selnot;
   assign selnot = ~sel;
   assign y = (a & selnot) | (b & sel);

endmodule
```

mux32x1.v:

```
module mux32x1(
              input wire [31:0] in,
              input wire [4:0] sel,
              output wire out
            );

   wire [15:0] level1;
   wire [7:0] level2;
```

```
    wire [3:0] level3;
    wire [1:0] level4;

    genvar i;
    generate
        for(i=0; i<16; i=i+1)begin
            mux2x1 M1(.a(in[i*2]), .b(in[i*2+1]), .sel(sel[0]), .y(level1[i]));
        end
        for(i=0; i<8; i=i+1)begin
            mux2x1 M2(.a(level1[2*i]), .b(level1[2*i+1]), .sel(sel[1]), .y(level2[i]));
        end
        for(i=0; i<4; i=i+1)begin
            mux2x1 M3(.a(level2[2*i]), .b(level2[2*i+1]), .sel(sel[2]), .y(level3[i]));
        end
        for(i=0; i<2; i=i+1)begin
            mux2x1 M4(.a(level3[2*i]), .b(level3[2*i+1]), .sel(sel[3]), .y(level4[i]));
        end
        mux2x1 M5(.a(level4[0]), .b(level4[1]), .sel(sel[4]), .y(out));
    endgenerate
endmodule
```

Description:

Copied from an earlier lab, this 32x1 mux is used as a selector for the different frequencies output by the clock divider

bcd_digit.v:

```
module bcd_digit(
                input clk,
                input rst_n,
                input en,
                input dir,
                output reg [3:0] digit,
                output wire rollover
            );

    assign rollover = en && ((dir && digit == 4'd9) || (!dir && digit == 4'd0));

    always@(posedge clk) begin
        if (!rst_n) begin
            digit <= 4'd0;
        end

        else if (en) begin
            if (dir) begin
                if (digit == 4'd9)
                    digit <= 4'd0;
                else
                    digit <= digit + 4'd1;
            end
            else begin
                if (digit == 4'd0)
                    digit <= 4'd9;
                else
                    digit <= digit - 4'd1;
```

```
        end
      end
   end
endmodule
```

bcd_up_down_counter.v:

```
module bcd_up_down_counter(
                        input clk,
                        input rst_n,
                        input dir,
                        output wire [7:0] BCD
   );

   wire [3:0] ones;
   wire [3:0] tens;
   wire rollover;

   bcd_digit ones_counter(
                            .clk(clk),
                            .rst_n(rst_n),
                            .en(1'b1),
                            .dir(dir),
                            .digit(ones),
                            .rollover(rollover)
                        );

   bcd_digit tens_counter(
                            .clk(clk),
                            .rst_n(rst_n),
                            .en(rollover),
                            .dir(dir),
                            .digit(tens),
                            .rollover()
                        );

  assign BCD = {tens, ones};

endmodule
```

Description:

This design for a cascasding up/down counter handles two BCD digits by cascading a single digit up/down counter with an enable for rollovers.

seg7_scan.v:

```
module seg7_scan(
                input clk,
                input rst_n,
                input [7:0] BCD,
                output reg [7:0] AN,
                output reg [6:0] SEG
            );
```

```
    reg [19:0] temp;
    reg [3:0] digit;

    wire s;
    assign s = temp[17];

    always@(posedge clk)
    begin
        if (!rst_n)
            temp <= 0;
        else
            temp <= temp + 1;
    end

    always@(*)
    begin
        case(s)
            1'b0: begin
                digit = BCD[3:0];
                AN = 8'b11111110;
            end

            1'b1: begin
                digit = BCD[7:4];
                AN = 8'b11111101;
            end

            default: begin
                digit = 0;
                AN = 8'b11111111;
            end
        endcase
    end

    always @(*)
    begin
        case(digit)
             4'd0: SEG=7'b0000001; 4'd1: SEG=7'b1001111; 4'd2: SEG=7'b0010010;
             4'd3: SEG=7'b0000110; 4'd4: SEG=7'b1001100; 4'd5: SEG=7'b0100100;
             4'd6: SEG=7'b0100000; 4'd7: SEG=7'b0001111; 4'd8: SEG=7'b0000000;
             4'd9: SEG=7'b0001100; default: SEG=7'b1111111;
         endcase
    end

endmodule
```

Description:

This seven segment controller time multiplexes bit 17 of a 20 bit sychronous delay register to drive two 7 segment displays and their digit patterns

debounce.v:

```
module debounce(
                input wire clk,
                input wire btn_raw,
```

```
            output reg btn_clean
        );

    reg[2:0] state;
    initial state = 3'b000;

    always @(posedge clk)begin
        state = {state[1:0], btn_raw};
        if(state == 3'b000) btn_clean = 0;
        else if (state == 3'b111) btn_clean = 1;
    end
endmodule
```

toggle_ff.v:

```
module toggle_ff (
                    input clk,
                    input rst_n,
                    input btn_raw,
                    output reg state
                  );
    wire btn_clean;
    reg btn_prev;

    debounce db (.clk(clk), .btn_raw(btn_raw), .btn_clean(btn_clean));

    always @(posedge clk) begin
        if (!rst_n) begin
            state <= 1;
            btn_prev <= 0;
        end
        else begin
            if (btn_clean && !btn_prev)
            state <= ~state;
            btn_prev <= btn_clean;
        end
    end
endmodule
```

Description:

These two modules were copied from lab3 for button implementation. Both button inputs are debounced and toggle is used for the direction select button U

Top_Lab5.v:

```
module top_lab5(
            input CLK100MHZ,
            input BTNC,
            input BTNU,
            input [4:0] SW,
            output wire [6:0] seg,
            output wire [7:0] an,
            output wire [12:0] LED
        );
```

```
    wire[31:0] counter;
    wire slow_clk;

    wire[7:0] BCD;

    wire rst;
    wire rst_n;

    wire dir;

    assign rst_n = ~rst;

    clock_divider clock_division(
                                .clk(CLK100MHZ),
                                .clk_div(counter)
                            );

    mux32x1 counter_multiplexer(
                                .in(counter),
                                .sel(~SW),
                                .out(slow_clk)
                            );

    debounce debounce_rst(
                            .btn_raw(BTNC),
                            .btn_clean(rst),
                            .clk(CLK100MHZ)
                        );

    toggle_ff toggle_dir(
                            .clk(CLK100MHZ),
                            .rst_n(rst_n),
                            .btn_raw(BTNU),
                            .state(dir)
                        );

    bcd_up_down_counter bcd_counter(
                                    .clk(slow_clk),
                                    .rst_n(rst_n),
                                    .dir(dir),
                                    .BCD(BCD)
                                );

    seg7_scan seg7_out(
                            .BCD(BCD),
                            .AN(an),
                            .SEG(seg),
                            .clk(CLK100MHZ),
                            .rst_n(rst_n)
                        );

   assign LED[4:0] = SW;
   assign LED[12:5] = BCD;

endmodule
```

Description:

The top module is very simple, it instantiates each module and wires them up. The only thing of note is the reversal of the sw input into the clock divider mux, this was done to match the given hardware interface where the speed select index is 0=slow, 31=fast (because the clock divider we generated has higher bits toggle at lower frequencies)

XDC Snippet:

```
 ## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { CLK100MHZ }];
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {CLK100MHZ}];


##Switches

set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { SW[0] }];
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { SW[1] }];
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { SW[2] }];
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { SW[3] }];
set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { SW[4] }];


## LEDs

set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { LED[0] }];
set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { LED[1] }];
set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { LED[2] }];
set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { LED[3] }];
…
set_property -dict { PACKAGE_PIN U14   IOSTANDARD LVCMOS33 } [get_ports { LED[10] }];
set_property -dict { PACKAGE_PIN T16   IOSTANDARD LVCMOS33 } [get_ports { LED[11] }];
set_property -dict { PACKAGE_PIN V15   IOSTANDARD LVCMOS33 } [get_ports { LED[12] }];


#7 segment display

set_property -dict { PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports {  seg[6] }]; #IO_L24N_T3_A00_D16_14 Sch=ca
set_property -dict { PACKAGE_PIN R10   IOSTANDARD LVCMOS33 } [get_ports {  seg[5] }]; #IO_25_14 Sch=cb
…
set_property -dict { PACKAGE_PIN T11   IOSTANDARD LVCMOS33 } [get_ports {  seg[1] }]; #IO_L19P_T3_A10_D26_14 Sch=cf
set_property -dict { PACKAGE_PIN L18   IOSTANDARD LVCMOS33 } [get_ports {  seg[0] }]; #IO_L4P_T0_D04_14 Sch=cg

set_property -dict { PACKAGE_PIN J17   IOSTANDARD LVCMOS33 } [get_ports { an[0] }]; #IO_L23P_T3_FOE_B_15 Sch=an[0]
set_property -dict { PACKAGE_PIN J18   IOSTANDARD LVCMOS33 } [get_ports { an[1] }]; #IO_L23N_T3_FWE_B_15 Sch=an[1]
…
set_property -dict { PACKAGE_PIN K2    IOSTANDARD LVCMOS33 } [get_ports { an[6] }]; #IO_L23P_T3_35 Sch=an[6]
set_property -dict { PACKAGE_PIN U13   IOSTANDARD LVCMOS33 } [get_ports { an[7] }]; #IO_L23N_T3_A02_D18_14 Sch=an[7]

##Buttons


set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { BTNC }]; #IO_L9P_T1_DQS_14 Sch=btnc
set_property -dict { PACKAGE_PIN M18   IOSTANDARD LVCMOS33 } [get_ports { BTNU }]; #IO_L4N_T0_D05_14 Sch=btnu
```

Simulation:

clock_divider_tb.v:

```
module clock_divider_tb (

                    );

    reg clk_tb;
    wire [31:0] clk_div_tb;

    integer i;
    integer count;  // Expected clk_div_tb count

    integer dn1, dn2; // delta nibble clk_div_tb[3:0], delta nibble clk_div_tb[7:4]
    integer clk_prev;   // clk_div_tb prev

    clock_divider dut (
                    .clk(clk_tb),
                    .clk_div(clk_div_tb)
                   );

    initial begin: Initialize
        clk_tb = 0;
        count = 0;
        dn1 = 0;
        dn2 = 0;
        clk_prev = 0;
    end

    always begin: Clock
        #5 clk_tb = ~clk_tb;
    end

    initial begin: Testbench
        #1
        for(i = 0; i < 128; i = i + 1) begin
            clk_prev = clk_div_tb[7:0];         // Set prev

            @(posedge clk_tb)
            count = count + 1;                  // Update count after clk edge

            #1
            if(count !== clk_div_tb) begin      // Fail if count !== clk_div_tb
                $display("FAIL  CLK = %b, CLK_DIV = %b", count, clk_div_tb);
                $fatal;
            end

            if(clk_div_tb[3:0] != clk_prev[3:0])    // Track delta nibble 1
                dn1 = dn1 + 1;

            if(clk_div_tb[7:4] != clk_prev[7:4])    // Track detla nibble 2
                dn2 = dn2 + 1;
        end
        #5

        $display("PASS  delta n1 = %d, delta n2 = %d", dn1, dn2);   // Expect dn1 to change 16* faster than dn2
```

```
    $finish;
  end

endmodule
```

Description:

To test the clock divider, I used a for loop to automatically verify it increments properly with each clock edge. I also kept track of how many times the first and second nibble of the clock divider changed to verify they changed at different frequencies. I found that the first nibble toggles 16 times faster than the second, confirming the implementation that higher bits toggle at lower frequencies.

Output:

```
run 12 ms
PASS  delta n1 =          128, delta n2 =            8
$finish called at time : 1281 ns : File "D:/School/ECE3300/ECE3300_Lab5_GroupU
```

Mux2x1_tb.v

```
module mux2x1_tb(

        );

  reg a, b, sel;
  wire y;
  integer tasksfailed, tempfailed;
  reg check;
  reg [1:0] i, j, k, in;

  mux2x1 mux_tb(.a(a), .b(b), .sel(sel), .y(y));

  initial begin
    a = 0;
    b = 0;
    sel = 0;
    tasksfailed = 0;
    tempfailed = 0;

    for(i=0; i<2; i=i+1)begin
      sel = i;
      for(j=0; j<2; j=j+1)begin
        a = j;
        for(k=0; k<2; k=k+1)begin
          b = k;
          in = {b, a};
          #10;
          check = y;
          if(check != in[sel]) tasksfailed = tasksfailed + 1;
          if(tasksfailed > tempfailed) $display("Test failed at %d, %d, %d", i, j, k);
```

```
                tempfailed = tasksfailed;
            end
        end
    end
    $finish;
  end
endmodule
```

Description:

This testbench uses nested for loops to check all four possible inputs for both possible selection values and checks the output.

Mux32x1_tb.v:

```
module mux32x1_tb(

            );

  reg[31:0] in;
  reg[4:0] sel;
  wire out;

  mux32x1 mux_tb(.in(in), .sel(sel), .out(out));

  integer i, tasksfailed;

  initial begin
    in = 32'b0;
    sel = 5'b0;
    tasksfailed = 0;
    for(i=0; i<32; i=i+1)begin
      in[i] = 1;
      #5;
      if(out != in[i]) begin
        tasksfailed = tasksfailed + 1;
        $display("test failed");
      end
      in[i] = 0;
      #5;
      if(out != in[i]) begin
        tasksfailed = tasksfailed + 1;
        $display("test failed");
      end
      sel = sel+1;
    end
    if(tasksfailed != 0) $display("%d tasks failed", tasksfailed); else $display("all tasks succeeded");
     $finish;
  end

endmodule
```

Description:

This testbench runs a nested four loop and checks that when the selected input is toggled that the output of the multiplexer does the same.

bcd_digit_tb.v:

```
module bcd_digit_tb(

                );

    reg clk_tb;
    reg rst_n_tb;
    reg en_tb;
    reg dir_tb;

    wire [3:0] digit_tb;
    wire rollover_tb;

    integer i;
    reg [3:0] expected;     // Expected digit value

    bcd_digit digit_test(
                    .clk(clk_tb),
                    .rst_n(rst_n_tb),
                    .en(en_tb),
                    .dir(dir_tb),
                    .digit(digit_tb),
                    .rollover(rollover_tb)
                   );

    initial begin
        clk_tb = 0;

        rst_n_tb = 0;
        en_tb = 1;
        dir_tb = 1;

        expected = 0;
    end

    always begin
        #5 clk_tb = ~clk_tb;
    end

    initial begin
        @(posedge clk_tb);  // Let digit settle after reset
        #1;
        rst_n_tb = 1;
        #1;
        if(digit_tb !== 4'd0) begin     // Test 1: Reset test
            $display("Fail: digit failed to settle on reset         rst_n = %b, digit = %d", rst_n_tb, digit_tb);
            $fatal;
        end
        for(i = 0; i < 10; i = i + 1) begin  // Test 2: Incrementing
            @(posedge clk_tb);
            if(expected == 4'd9) begin  // Rollover control for expected value
                expected = 4'd0;
            end
```

```
    else begin
        expected = expected + 4'd1;
    end
    #1;
    if(digit_tb !== expected) begin  // Digit test
        $display("Fail: Digit did not increment properly        digit = %d, expected = %d", digit_tb, expected);
        $fatal;
    end
    if(digit_tb == 4'd9) begin
        if(rollover_tb !== 1'b1) begin // Rollover signal test
            $display("Fail: Rollover signal did not activate properly       digit = %d, rollover = %d", digit_tb, rollover_tb);
            $fatal;
        end
    end
end

expected = 4'd1;    // Account for extra up count before switching to down

@(posedge clk_tb);
#1;
dir_tb = 0;
for(i = 0; i < 12; i = i + 1) begin  // Test 3: Decrementing
    @(posedge clk_tb);
    if(expected == 4'd0) begin  // Rollover control for expected value
        expected = 4'd9;
    end
    else begin
        expected = expected - 4'd1;
    end
    #1;
    if(digit_tb !== expected) begin  // Digit test
        $display("Fail: Digit did not decrement properly        digit = %d, expected = %d", digit_tb, expected);
        $fatal;
    end
    if(digit_tb == 4'd0) begin
        if(rollover_tb !== 1'b1) begin // Rollover signal test
            $display("Fail: Rollover signal did not activate properly       digit = %d, rollover = %d", digit_tb, rollover_tb);
            $fatal;
        end
    end
end

@(posedge clk_tb);
#1;
en_tb = 0;
@(posedge clk_tb);
#1;
if(digit_tb !== 4'd8) begin     // Test 4: Enable test
$display("Fail: Operated while enable low");
$fatal;
end

@(posedge clk_tb);
#1;
rst_n_tb = 0;               // Test 5: Final reset test
@(posedge clk_tb);
#3;
if(digit_tb !== 4'd0) begin
    $display("Fail: Reset did not reset to zero     digit = %d, rst_n = %b", digit_tb, rst_n_tb);
    $fatal;
end
```

```
    @(posedge clk_tb);
    $display("Passed all tests");
    $finish;
  end

endmodule
```

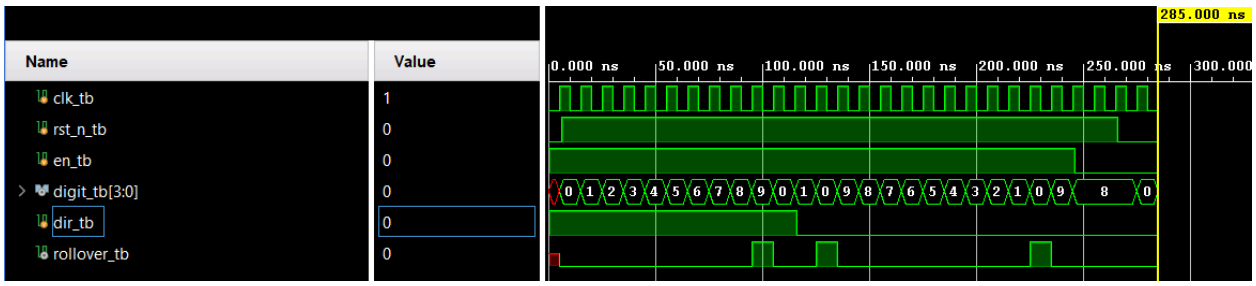
Description:

To test the single digit up/down counter, I used an expected value and automatic checks to verify the counter increments and decrements properly with a rollover signal at 9 for incrementing and 0 for decrementing. I also tested that the digit does not change when enable is low goes to zero on reset.

Output:

```
Passed all tests
$finish called at time : 285 ns : File "D:/School/ECE3300/ECE3300_Lab5_GroupU
```

Waveform:



bcd_up_down_counter_tb.v:

```
module bcd_up_down_counter_tb(

                    );

  reg clk_tb;
  reg rst_n_tb;
  reg dir_tb;
  wire [7:0] BCD_tb;


  integer i;
  integer expected;          // Expected decimal value

  reg [3:0] expected_tens;    // Expected tens value
  reg [3:0] expected_ones;    // Expected ones value

  reg [7:0] expected_bcd;     // Expected BCD value {tens, ones}
```

```
bcd_up_down_counter bcd_counter(
                                .clk(clk_tb),
                                .rst_n(rst_n_tb),
                                .dir(dir_tb),
                                .BCD(BCD_tb)
                            );

initial begin
    clk_tb = 0;
    rst_n_tb = 0;
    dir_tb = 1;
    expected = 0;
    expected_bcd = 8'h00;
end

always begin
    #5 clk_tb = ~clk_tb;
end

initial begin
    @(posedge clk_tb);  // Let BCD settle after reset
    #1;
    rst_n_tb = 1;
    #1;
    if(BCD_tb !== 8'h00) begin     // Test 1: Reset test
        $display("Fail: BCD failed to settle on reset         rst_n = %b, BCD = %h", rst_n_tb, BCD_tb);
        $fatal;
    end
    for(i = 0; i < 100; i = i + 1) begin  // Test 2: Incrementing
        @(posedge clk_tb);
        if(expected == 99) begin  // Rollover control for expected value
            expected = 0;
        end
        else begin
            expected = expected + 1;
        end

        expected_tens = expected / 10;
        expected_ones = expected % 10;
        expected_bcd = {expected_tens, expected_ones};

        #1;
        if(BCD_tb !== expected_bcd) begin
            $display("Fail: BCD did not increment properly        BCD = %h, expected = %h", BCD_tb, expected_bcd);
            $fatal;
        end
    end

    expected = 1;   // Account for extra up count before switching to down

    @(posedge clk_tb);
    #1;
    dir_tb = 0;
    for(i = 0; i < 105; i = i + 1) begin  // Test 3: Decrementing
        @(posedge clk_tb);
        if(expected == 0) begin  // Rollover control for expected value
            expected = 99;
        end
        else begin
            expected = expected - 1;
```

```
        end

        expected_tens = expected / 10;
        expected_ones = expected % 10;
        expected_bcd = {expected_tens, expected_ones};

        #1;
        if(BCD_tb !== expected_bcd) begin
            $display("Fail: BCD did not decrement properly        BCD = %h, expected = %h", BCD_tb, expected_bcd);
            $fatal;
        end
    end

    @(posedge clk_tb);
    #1;
    rst_n_tb = 0;          // Test 4: Final reset test
    @(posedge clk_tb);
    #1;
    if(BCD_tb !== 8'h00) begin
        $display("Fail: Reset did not reset to zero     BCD = %h, rst_n = %b", BCD_tb, rst_n_tb);
        $fatal;
    end

    @(posedge clk_tb);
    $display("Passed all tests");
    $finish;
end

endmodule
```

Description:

This module is very similar to the previous but with higher bounds and a check in BCD. The expected value is converted into two bcd digits for tens and ones and concatinated for this. There is also a reset check.

Output:

```
run 12 ms
Passed all tests
$finish called at time : 2095 ns : File "D:/School/ECE3300/ECE3300_Lab5_GroupU
```

seg7_scan_tb.v:

```
module seg7_scan_tb(

            );

    reg clk_tb;
    reg rst_n_tb;
    reg [7:0] BCD_tb;
```

```
wire [7:0] AN_tb;
wire [6:0] SEG_tb;

integer i;
integer tasksfailed;
integer tempfail;

reg [6:0] displaycheck;

seg7_scan scan_tb(
                .clk(clk_tb),
                .rst_n(rst_n_tb),
                .BCD(BCD_tb),
                .AN(AN_tb),
                .SEG(SEG_tb)
              );

initial
begin
   clk_tb = 1'b0;
   rst_n_tb = 1'b0;
   BCD_tb = 8'h00;

   tasksfailed = 0;
   tempfail = 0;
   displaycheck = 7'b1111111;
end

always
begin
   #5 clk_tb = ~clk_tb;
end

initial
begin
   @(posedge clk_tb);
   #1;
   rst_n_tb = 1'b1;

   for(i = 0; i < 10; i = i + 1)
   begin
      BCD_tb = {i[3:0], i[3:0]};

      tempfail = tasksfailed;

      case(i)
            0: displaycheck = 7'b0000001;
            1: displaycheck = 7'b1001111;
            2: displaycheck = 7'b0010010;
            3: displaycheck = 7'b0000110;
            4: displaycheck = 7'b1001100;
            5: displaycheck = 7'b0100100;
            6: displaycheck = 7'b0100000;
            7: displaycheck = 7'b0001111;
            8: displaycheck = 7'b0000000;
            9: displaycheck = 7'b0001100;
            default: displaycheck = 7'b1111111;
      endcase

      #40;
```

```
        if(displaycheck !== SEG_tb)
        begin
            tasksfailed = tasksfailed + 1;
        end

        if(tasksfailed > tempfail)
        begin
            $display("SEG test failed at i = %d, SEG = %b, expected = %b", i, SEG_tb, displaycheck);
        end
    end

    if(tasksfailed == 0)
        begin
            $display("SEG tests passed");
        end
    else
        begin
            $display("%d SEG tests failed", tasksfailed);
        end

    #5000000;
    $finish;
end

endmodule
```

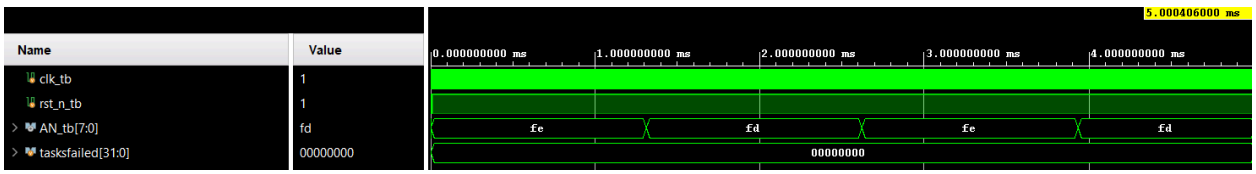
Description:

To test the 7segment display driver, each number is automatically checked against its corrosponding digit pattern. After this, the testbench runs for 5ms to show 4 instances of the AN display switching for scan multiplexing (Since the driver "s" is defined by [19:17] of the tmp register that increments every cycle, s change after 131,072 cycles. With my clock running at 10ms cycles, AN switches every ~1.32ms)

Output:

```
run 12 ms
SEG tests passed
$finish called at time : 5000406 ns : File "D:/School/ECE3300/ECE3300_Lab5_GroupU
```

Waveform:



debounce_tb.v:

```
module debounce_tb(

                );

    reg clk_tb;
    reg btn_raw_tb;
    wire btn_clean_tb;

    debounce db_tb(
                  .clk(clk_tb),
                  .btn_raw(btn_raw_tb),
                  .btn_clean(btn_clean_tb)
                );

    initial
       begin
          clk_tb = 1'b0;
       end

    always
       begin
          #5 clk_tb = ~clk_tb;
        end

     initial
       begin
          btn_raw_tb = 1'b0;                // Test 1: Stable low
          repeat(5) @(posedge clk_tb);
          #1;

          if(btn_clean_tb !== 1'b0)
          begin
             $display("FAIL: btn_clean should be 0 after stable low");
             $fatal;
          end

          btn_raw_tb = 1'b1;                // Test 2: Bounce
          @(posedge clk_tb);
          #1;

          btn_raw_tb = 1'b0;
          @(posedge clk_tb);
          #1;

          if(btn_clean_tb !== 1'b0)
          begin
             $display("FAIL: btn_clean changed after bounce");
             $fatal;
          end

          btn_raw_tb = 1'b1;                // Test 3: Stable high
          repeat(5) @(posedge clk_tb);
          #1;

          if(btn_clean_tb !== 1'b1)
          begin
             $display("FAIL: btn_clean should be 1 after stable high");
             $fatal;
```

```
        end

        btn_raw_tb = 1'b0;              // Test 4: Bounce
        @(posedge clk_tb);
        #1;

        btn_raw_tb = 1'b1;
        @(posedge clk_tb);
        #1;

        if(btn_clean_tb !== 1'b1)
        begin
           $display("FAIL: btn_clean changed after bounce");
           $fatal;
        end

        btn_raw_tb = 1'b0;              // Test 5: Final stable low
        repeat(5) @(posedge clk_tb);
        #1;

        if(btn_clean_tb !== 1'b0)
        begin
           $display("FAIL: btn_clean should be 0 after stable low");
           $fatal;
        end

        $display("Passed all tests");
        $finish;
    end
endmodule
```

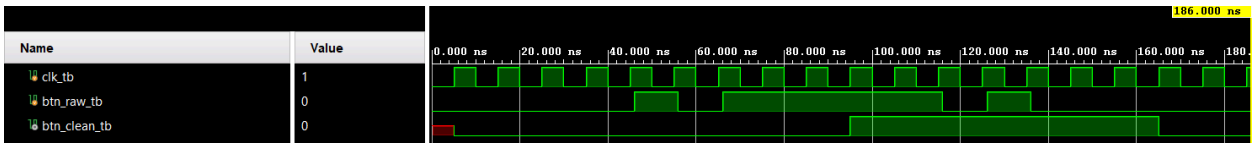
Description:

This debounce testbench automatically verifies the output of stable low -> bouncing -> stable high -> bouncing -> stable low

Output:

```
Passed all tests
$finish called at time : 186 ns : File "D:/School/ECE3300/ECE3300 Lab5 GroupU
```

Waveform:



toggle_ff_tb.v:

```
module toggle_ff_tb(
```

```
            );

reg clk_tb;
reg rst_tb;
reg btn_raw_tb;
wire state_tb;

integer i;

toggle_ff toggle_tb(
                    .clk(clk_tb),
                    .rst_n(rst_tb),
                    .btn_raw(btn_raw_tb),
                    .state(state_tb)
                );

initial
   begin
      clk_tb = 1'b0;
   end

always
   begin
      #5 clk_tb = ~clk_tb;
   end

initial
      begin
         rst_tb = 1'b1;
         btn_raw_tb = 1'b0;

         for(i = 0; i < 3; i = i + 1)
            begin
               @(posedge clk_tb);
            end

         @(negedge clk_tb);
         rst_tb = 1'b0;
         btn_raw_tb = 1'b1;


         for(i = 0; i < 6; i = i + 1)
            begin
               @(posedge clk_tb);
            end

            @(negedge clk_tb);

         if(state_tb !== 1'b1)
            begin
               $display("FAILED TO TOGGLE");
               $fatal;
            end
         else
            begin
               $display("PASSED TEST TOGGLE");
            end

         @(negedge clk_tb);
         rst_tb = 1'b1;
         btn_raw_tb = 1'b0;
```

```
            @(negedge clk_tb);
            if(state_tb !== 1'b0)
               begin
                  $display("FAILED TO RESET");
                  $fatal;
               end
            else
               begin
                  $display("PASSED TEST RESET");
               end
            $finish;
         end
endmodule
```

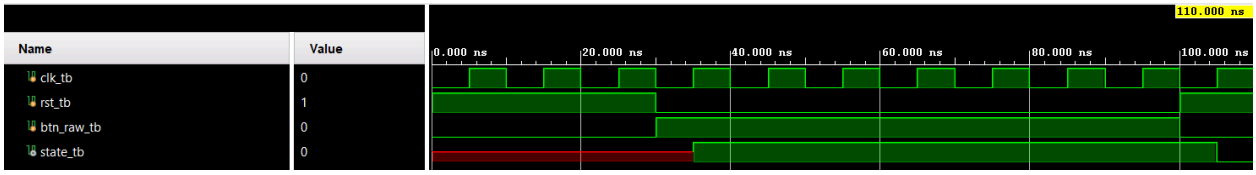
Description:

To verify the toggle switch implementation i reset the state initially, let the debounce output settle, and activated the toggle. Within a few cycles, I verified that the state changed. After this, I activated the reset button again, verifying that the state goes back to low. Each section is self checking with a console output and failure breaks.

Output:

```
PASSED TEST TOGGLE
PASSED TEST RESET
$finish called at time : 110 ns : File "D:/School/ECE3300/ECE3300_Lab5_GroupU
```

Waveform:



top_lab5_tb.v:

```
module top_lab5_tb(

            );

   reg clk_tb;
   reg btnc_tb;
   reg btnu_tb;
   reg [4:0] sw_tb;

   wire [6:0] seg_tb;
   wire [7:0] an_tb;
   wire [12:0] led_tb;

   integer i;
```

```
    top_lab5 top_tb(
                    .CLK100MHZ(clk_tb),
                    .BTNC(btnc_tb),
                    .BTNU(btnu_tb),
                    .SW(sw_tb),
                    .seg(seg_tb),
                    .an(an_tb),
                    .LED(led_tb)
                );

    initial
    begin
        clk_tb = 0;
    end

    always
    begin
        #5 clk_tb = ~clk_tb;
    end

    initial
    begin
        btnc_tb = 0;
        btnu_tb = 0;
        sw_tb = 0;

        for(i = 0; i < 32; i = i + 1) begin     // Test LED[4:0] = SW
            sw_tb = i[4:0];
            #50;

            if(led_tb[4:0] !== sw_tb) begin
                $display("Fail: LED[4:0] != SW      LED = %b, SW = %b", led_tb, sw_tb);
                $fatal;
            end
        end

        $display("Passed LED test");

        btnc_tb = 1;
        #500;
        btnc_tb = 0;
        #5000000;

        $finish;
    end

endmodule
```

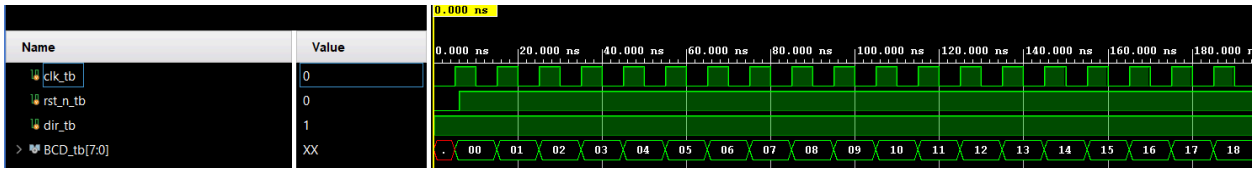
Description:

Since all the other modules were tested individually and the top module only functions to wire them, I focused on verifying the LED implementation. Automatic checks show LED[4:0] = SW inputs, and zooming into the waveform shows the LED value incrementing in BCD. Additionally, this re-tests the previously tested button functionality

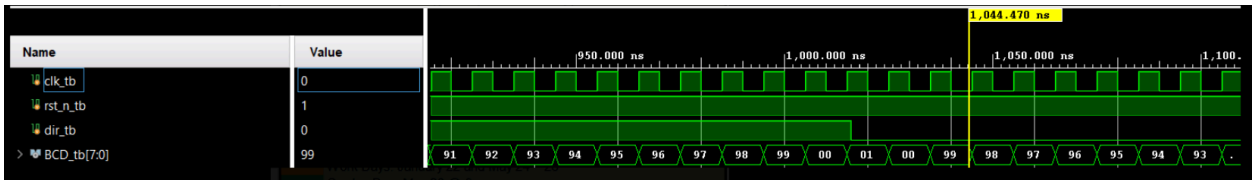
Output:

```
Passed LED test
$finish called at time : 5002100 ns : File "D:/School/ECE3300/ECE3300_Lab5_GroupU
```

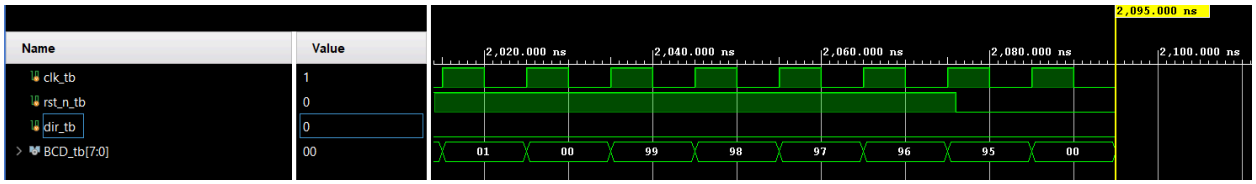
Waveform:



BCD incrementing



Increment Rollover



Decrement Rollover

Implementation:

Resource Utilization Report:

Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	Bonded IOB (210)	BUFGCTRL (32)
N top_lab5	30	66	1	36	1

Timing Report:

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	6.851 ns	Worst Hold Slack (WHS):	0.137 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	76	Total Number of Endpoints:	76	Total Number of Endpoints:	59

All user specified timing constraints are met.

Group Video:

https://youtu.be/daCQe4b1Tb8

Contributions:

Ben Robles: Counter design & testbench/simulation
Robert Stevenson: Core design and testbenches

Reflections:

Robert Stevenson: This lab was interesting and challenging. This one required a more complicated top level module than the previous labs.

Ben Robles: In this lab I really enjoyed working in RTL and having the freedom to design the project. This made testbenching easier as I fully understood each module I worked with hands on.